

Manacher

Manacher

描述

给定一个长度为 n 的字符串，请找到所有对 i 使得子串 $s[i-l:i+l]$ 为一个回文串。当 $l=0$ 时，字符串 s 是一个回文串（ s 是 s 的反转字符串）。

解释

显然在最坏情况下可能有 n 个回文串，因此似乎一眼看过去该问题并没有线性算法。

但是关于回文串的信息可用 **一种更紧凑的方式** 表达：对于每个位置 i ，我们找出值 l_i 和 r_i 。二者分别表示以位置 i 为中心的长度为奇数和长度为偶数的回文串个数。换个角度，二者也表示了以位置 i 为中心的最长回文串的半径长度（半径长度 l_i ， r_i 均为从位置 i 到回文串最右端位置包含的字符个数）。

举例来说，字符串 $abacabad$ 以 a 为中心有三个奇数长度的回文串，最长回文串半径为 3 ，也即：

字符串 $abacabad$ 以 b 为中心有两个偶数长度的回文串，最长回文串半径为 2 ，也即：

因此关键思路是，如果以某个位置 i 为中心，我们有一个长度为 l_i 的回文串，那么我们有以 i 为中心的半径为 $1, 2, \dots, l_i$ 的回文串。所以 l_i 和 r_i 两个数组已经足够表示字符串中所有子回文串的信息。

一个令人惊讶的事实是，存在一个复杂度为线性并且足够简单的算法计算上述两个「回文性质数组」 l_i 和 r_i 。在这篇文章中我们将详细的描述该算法。

解法

总的来说，该问题具有多种解法：应用字符串哈希，该问题可在 $O(n)$ 时间内解决，而使用后缀数组和快速 LCA 该问题可在 $O(n \log n)$ 时间内解决。

但是这里描述的算法 **压倒性** 的简单，并且在时间和空间复杂度上具有更小的常数。该算法由 **Glenn K. Manacher** 在 1975 年提出。

朴素算法

为了避免在之后的叙述中出现歧义，这里我们指出什么是「朴素算法」。

该算法通过下述方式工作：对每个中心位置 i ，在比较一对对应字符后，只要可能，该算法便尝试将答案加 1 。

该算法是比较慢的：它只能在 $O(n^2)$ 的时间内计算答案。

该朴素算法的实现如下：

▼ 实现



C++Python

```

1 vector<int> d1(n), d2(n);
2 for (int i = 0; i < n; i++) {
3     d1[i] = 1;
4     while (0 <= i - d1[i] && i + d1[i] < n && s[i - d1[i]] == s[i + d1[i]]) {
5         d1[i]++;
6     }
7
8     d2[i] = 0;
9     while (0 <= i - d2[i] - 1 && i + d2[i] < n &&
10         s[i - d2[i] - 1] == s[i + d2[i]]) {
11         d2[i]++;
12     }
13 }

1 d1 = [0] * n
2 d2 = [0] * n
3 for i in range(0, n):
4     d1[i] = 1
5     while 0 <= i - d1[i] and i + d1[i] < n and s[i - d1[i]] == s[i + d1[i]]:
6         d1[i] += 1
7
8     d2[i] = 0
9     while 0 <= i - d2[i] - 1 and i + d2[i] < n and s[i - d2[i] - 1] == s[i + d2[i]]:
10         d2[i] += 1

```

Manacher 算法

这里我们将只描述算法中寻找所有奇数长度子回文串的情况，即只计算 $d1$ ；寻找所有偶数长度子回文串的算法（即计算 $d2$ ）将只需对奇数情况下的算法进行一些小修改。

为了快速计算，我们维护已找到的最靠右的子回文串的 **边界**（即具有最大 $d1$ 值的回文串，其中 i 和 $i + d1[i]$ 分别为该回文串左右边界的位置）。初始时，我们置 $i = 0$ 和 $i + d1[i] = -1$ （-1 需区别于倒序索引位置，这里可为任意负数，仅为了循环初始时方便）。

过程

现在假设我们要对下一个 i 计算，而之前所有 i 中的值已计算完毕。我们将通过下列方式计算：

- 如果 i 位于当前子回文串之外，即 $i > i + d1[i]$ ，那么我们调用朴素算法。

因此我们将连续地增加 i ，同时在每一步中检查当前的子串（ i 表示半径长度，下同）是否为一个回文串。如果我们找到了第一处对应字符不同，又或者碰到了 $i + d1[i]$ 的边界，则算法停止。在两种情况下我们均已计算完 $d1[i]$ 。此后，仍需记得更新 $i + d1[i]$ 。

- 现在考虑 $i \leq i + d1[i]$ 的情况。我们将尝试从已计算过的 i 的值中获取一些信息。首先在子回文串 i 中反转位置 i ，即我们得到 $i + d1[i]$ 。现在来考察值 i 。因为位置 i 同位置 $i + d1[i]$ 对称，我们 **几乎总是** 可以置 $d1[i] = d1[i + d1[i]]$ 。该想法的图示如下（可认为以 $i + d1[i] / 2$ 为中心的回文串被「拷贝」至以 i 为中心的位置上）：

然而有一个 **棘手的情况** 需要被正确处理：当「内部」的回文串到达「外部」回文串的边界时，即 $i + d1[i] = i + d1[i + d1[i]]$ （或者等价的说， $i + d1[i] = i + d1[i + d1[i]]$ ）。因为在「外部」回文串范围以外的对称性没有保证，因此直接置 $d1[i] = d1[i + d1[i]]$ 将是不正确的：我们没有足够的信息来断言在位置 i 的回文串具有同样的长度。

实际上，为了正确处理这种情况，我们应该「截断」回文串的长度，即置 $d1[i] = d1[i + d1[i]]$ 。之后我们将运行朴素算法以尝试尽可能增加 $d1[i]$ 的值。

该种情况的图示如下（以 $i + d1[i] / 2$ 为中心的回文串已经被截断以落在「外部」回文串内）：

该图示显示出，尽管以 `l` 为中心的回文串可能更长，以致于超出「外部」回文串，但在位置 `i`，我们只能利用其完全落在「外部」回文串内的部分。然而位置 `i` 的答案可能比这个值更大，因此接下来我们将运行朴素算法来尝试将其扩展至「外部」回文串之外，也即标识为 "try moving here" 的区域。

最后，仍有必要提醒的是，我们应当记得在计算完每个 `i` 后更新值 `l`。

同时，再让我们重复一遍：计算偶数长度回文串数组 `dl` 的算法同上述计算奇数长度回文串数组 `dl` 的算法十分类似。

Manacher 算法的复杂度

因为在计算一个特定位置的答案时我们总会运行朴素算法，所以一眼看去该算法的时间复杂度为线性的事实并不显然。

然而更仔细的分析显示出该算法具有线性复杂度。此处我们需要指出，[计算 Z 函数的算法](#) 和该算法较为类似，并同样具有线性时间复杂度。

实际上，注意到朴素算法的每次迭代均会使 `l` 增加，以及 `r` 在算法运行过程中从不减小。这两个观察告诉我们朴素算法总共会进行 $O(n)$ 次迭代。

Manacher 算法的另一部分显然也是线性的，因此总复杂度为 $O(n)$ 。

Manacher 算法的实现

分类讨论

为了计算 `dl`，我们有以下代码：



C++Python

```
1 vector<int> dl(n);
2 for (int i = 0, l = 0, r = -1; i < n; i++) {
3     int k = (i > r) ? 1 : min(dl[l + r - i], r - i + 1);
4     while (0 <= i - k && i + k < n && s[i - k] == s[i + k]) {
5         k++;
6     }
7     dl[i] = k--;
8     if (i + k > r) {
9         l = i - k;
10        r = i + k;
11    }
12 }
```

```
1 dl = [0] * n
2 l, r = 0, -1
3 for i in range(0, n):
4     k = 1 if i > r else min(dl[l + r - i], r - i + 1)
5     while 0 <= i - k and i + k < n and s[i - k] == s[i + k]:
6         k += 1
7     dl[i] = k
8     k -= 1
9     if i + k > r:
10        l = i - k
11        r = i + k
```

计算 `zl` 的代码十分类似，但是在算术表达式上有些许不同：



C++Python

```
1 vector<int> d2(n);
2 for (int i = 0, l = 0, r = -1; i < n; i++) {
3     int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
4     while (0 <= i - k - 1 && i + k < n && s[i - k - 1] == s[i + k]) {
5         k++;
6     }
7     d2[i] = k--;
8     if (i + k > r) {
9         l = i - k - 1;
10        r = i + k;
11    }
12 }
```

```
1 d2 = [0] * n
2 l, r = 0, -1
3 for i in range(0, n):
4     k = 0 if i > r else min(d2[l + r - i + 1], r - i + 1)
5     while 0 <= i - k - 1 and i + k < n and s[i - k - 1] == s[i + k]:
6         k += 1
7     d2[i] = k
8     k -= 1
9     if i + k > r:
10        l = i - k - 1
11        r = i + k
```

统一处理

虽然在讲解过程及上述实现中我们将 和 的计算分开考虑，但实际上可以通过一个技巧将二者的计算统一为 的计算。

给定一个长度为 的字符串，我们在其 个空中插入分隔符，从而构造一个长度为 的字符串。举例来说，对于字符串，其对应的。

对于字母间的，其实际意义为 中对应的「空」。而两端的 则是为了实现的方便。

注意到，在对 计算后，对于一个位置，所描述的最长的子回文串必定以 结尾（若以字母结尾，由于字母两侧必定各有一个，因此可向外扩展一个得到一个更长的）。因此，对于 中一个以字母为中心的极大子回文串，设其长度为，则其在 中对应一个以相应字母为中心，长度为 的极大子回文串；而对于 中一个以空为中心的极大子回文串，设其长度为，则其在 中对应一个以相应表示空的 为中心，长度为 的极大子回文串（上述两种情况下的 均为偶数，但该性质成立与否并不影响结论）。综合以上观察及少许计算后易得，在 中，表示在 中以对应位置为中心的极大子回文串的总长度加一。

上述结论建立了 的 同 的 和 间的关系。

由于该统一处理本质上即求 的，因此在得到 后，代码同上节计算 的一样。

练习题目

- [UVa #11475 "Extend to Palindrome"](#)
- [「国家集训队」最长双回文串](#)

本页面最近更新：2024/5/8 20:33:33, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[iamtwz](#), [abc1763613206](#), [Alisahhh](#), [aofall](#), [CamberLoid](#), [cesonic](#), [chenhongqiao](#), [CoelacanthusHex](#), [Enter-tainer](#), [fseasy](#), [gi-b716](#), [Henry-ZHR](#), [lr1d](#), [ksyx](#), [LeoJacob](#), [Marcythm](#), [Menci](#), [ouuan](#), [pengxurui](#), [Persdre](#), [shawlleyw](#), [shuzhouliu](#), [sshwy](#), [StudyingFather](#), [Tiphereth-A](#), [TrisolarisHD](#), [Wsuika](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用